

# **COMPENG 2DX3**

## **Final Project Report**

04/07/2026  
Hongfan Li

As a future member of the engineering profession, the student is responsible for performing the required work in an honest manner, without plagiarism and cheating. Submitting this work with my name and student number is a statement and understanding that this work is my own and adheres to the Academic Integrity Policy of McMaster University and the Code of Conduct of the Professional Engineers of Ontario. Submitted by **[Hongfan Li]**

## Device Overview – Features

The device is a combination of several components, and each component has its own features. In total, the cost of the device is around 250 Canadian dollars.

### **MSP-EXP432E401Y Microcontroller**

- Has a 32-bit Cortex-M4F core processor
- Assigned bus speed 10 MHz
- GPIOs for inputs and outputs
- On-board LEDs to indicate different status
- On-board push buttons to control the program
- 1 MB Flash memory and 256 KB SRAM

### **28BYJ-48 Stepper Motor**

- Require a 5 – 12 V voltage supply
- Has 4 on-board LEDs to indicate the current state
- Require 2048 steps to rotate 360 degrees in full-step mode
- Steps are in a sequence of 4 and keep repeating

### **VL53L1X Time of Flight Sensor**

- Maximum distance measuring 4 m
- 2.6 – 3.5 V voltage supply

### **Communication protocols**

- I2C is used for data transmission between the microcontroller and the ToF sensor
- UART is used for data transmission between the microcontroller and the PC

### **3-D Visualization**

- MATLAB is used to turn the distance data into 3-D (x, y, z)
- Then MATLAB plot a 3-D diagram based on the 3-D data.

## Device Overview – General Description

The system is very straightforward and easy-to-use 3-D visualization of the internal structure of certain space. The system combined the microcontroller, the ToF sensor, the stepper motor and the PC (use the MATLAB). The I2C is used for communication between the ToF sensor and the microcontroller and PB2 and PB3 pin on the microcontroller is connected to the SCL and SDA pin on the ToF sensor. The PH0 - PH3 is connected to the stepper motor driver. The UART is used for the communication between the microcontroller and the PC.

The on-board push button PJ0 is used to start 10 scans. The on-board LED D4 will flash when a measurement is taking place, the on-board LED D3 will flash when a UART transmission is taking place, and the on-board LED D1 will be on when the motor is working otherwise it will be off.

For the main program. After the reset button is pressed, the program will initialize PLL, SysTick, all the ports, I2C, UART, and the sensor. Then after pressing the PJ0, it will start the 10 scans and use UART to send the state and distance data to the MATLAB. After each scan, it will rotate back to the starting point because it can avoid the wires tangled up. After all the scans all finished, it will call the Transfer\_data() function which will send the complete data which contains the layer of scan, degree, distance and x\_displacement to the MATLAB. Finally, the MATLAB will turn the data into 3-D point and create the 3-D plot which connect all the point in the same layer and in the same measurement degree.

## Device Overview – Block Diagram

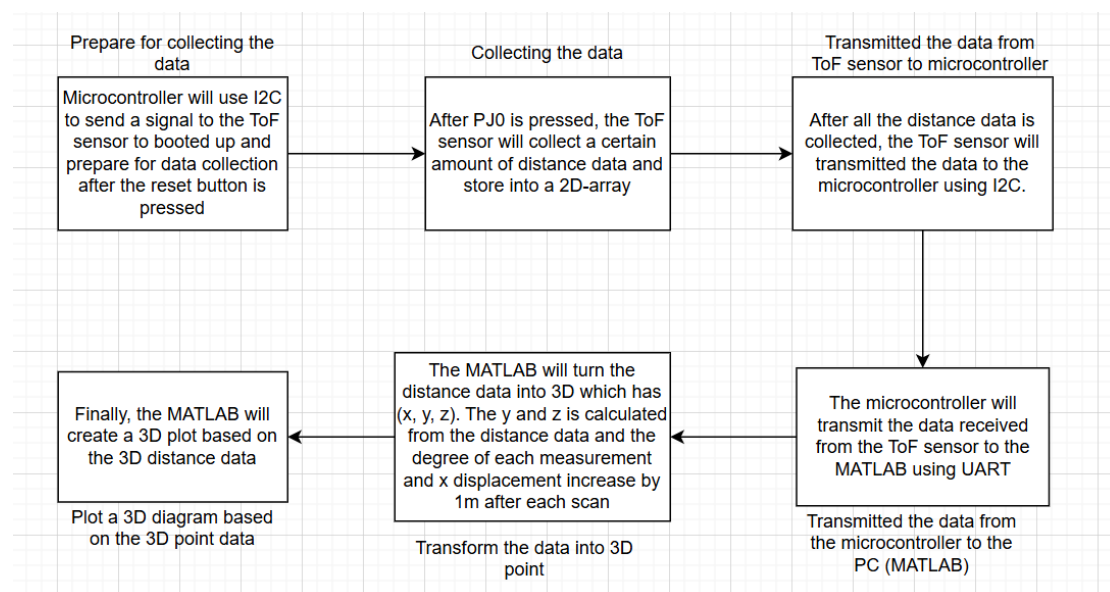


Figure 1: the block diagram

## Device Characteristics Table

| Microcontroller              |       |
|------------------------------|-------|
| Clock speed                  | 10MHz |
| Measurement status LED       | PF0   |
| UART transmission status LED | PF4   |
| Motor working status LED     | PN1   |
| Start button                 | PJ0   |

|                                      |               |
|--------------------------------------|---------------|
| Serial Port                          | COM4          |
| Baud Rate                            | 115200 BPs    |
| <b>Stepper Motor Driver</b>          |               |
| IN1                                  | PH0           |
| IN2                                  | PH1           |
| IN3                                  | PH2           |
| IN4                                  | PH3           |
| V+                                   | 5V            |
| V-                                   | GND           |
| <b>Time of Flight (ToF) Sensor</b>   |               |
| VIN                                  | 3.3V          |
| GND                                  | GND           |
| SDA                                  | PB3           |
| SCL                                  | PB2           |
| <b>Device working characteristic</b> |               |
| Number of measurements per scan      | 32            |
| The x displacement between each scan | 1m            |
| Rotation between each measurement    | 11.25 degrees |

Table 1: the device characteristic table

## Detailed Description – Distance Measurement

For the VL53L1X Time of Flight sensor, it works by emitting a light which will collide with the obstacle, then it will return back to the sensor. The distance to the obstacle can be measured by measuring the time for the light from emit to return back. Since the light goes through double the distance to the obstacle. Therefore, the distance is equal to  $distance = \frac{speed\ of\ the\ light * time\ taken\ for\ the\ light\ to\ come\ back}{2}$ .

After getting this value, it will do transduction, signal conditioning and analog to digital conversion. Then transmitted the data to the microcontroller using I2C where PB3 is connected to SDA which is the data bit and PB2 is connected to the SCL which is the clock.

In the main program, the ToF sensor is initialized after the reset button is pressed. It will boot the sensor and prepare to measure the distance. Then in my C program, I use the polling method to keep checking the button press of PJ0, the measurement only starts after the button is pressed. Also, there is a on-board LED D4 which will blink when the measurement is taking place.

When the measurement starts, there is a for loop which controls the number of scans which is 10. Different scan just creates a different layer of points since there is 1m displacement between each scan. It is implemented by just using  $layer * 1000$ .

Since the layer starts from 0 which means start from displacement equal to 0 and the layer increases by 1 for each scan. Also, the ToF sensor measures the distance in units of mm. Therefore, we use the layer times 1000 to get the x displacement in mm. Within each scan, 32 measurements are taking place. It means after a measurement taking place, the stepper motor will rotate 11.25 degrees before the next measurement starts. Therefore, the stepper motor needs to move 64 steps clockwise after each measurement because one whole rotation requires 2048 steps. So, through the calculation  $steps = \frac{11.25}{360} * 2048 = 64$ . And after 32 measurement is done, the stepper motor will rotate counterclockwise which move back to the starting position which avoid the wire to tangled up and ready for the next scan.

```
uint16_t distance[3][32];
```

Figure 2: the global variable in Keil c program to store the distance data measured

After each measurement, the distance data is stored into the 2D array created as a global variable, like distance[10][32]. It will store the distance data for each scan into a sub array. When a transmission is taking place, the on-board LED D3 will flash. After these data is transmitted to the MATLAB using UART. It will check the position of the data which the first element in the sub array means the angle is 0 and increases 11.25 degree for the rest of the elements. To turn the distance data into y, z, calculation are used:  $y = distance * \sin(angle\_data)$  and  $z = distance * \cos(ang el\_data)$ . And x is just equal to the x displacement calculated before.

```
x_positions = x_mm / 1000;           % mm to m
r = distance_mm / 1000;             % mm to m
theta = deg2rad(angle_deg);

Y = r .* cos(theta);
Z = r .* sin(theta);
X = x_positions;
```

Figure 3: the MATLAB code for turning the distance input into y, z coordinates.

## Detailed Description – Visualization

After all the scans are done, the data will be transmitted to the MATLAB using UART. The MATLAB need to connect to the COM4 port and set the baud rate to 115200 for communication. In the MATLAB, it will resort all the data (displayed in [layer, x\_dispalcement, angle, distance]) by splitting in columns. Then we will get the x, y and z value based on the calculation steps mentioned above. Also, we are going to change the unit from mm to m which will divide all the numbers by 1000.

```

scan_num = data(:,1);
x_mm = data(:,2);
angle_deg = data(:,3);
distance_mm = data(:,4);

```

*Figure 4: MATLAB code that splits the data in columns*

First, I am going to plot the scan planes connected with a closed line for each plane, I create a closed loop for each scan by repeating the first element. Then I can plot the scans in 3D with the points in the same scan connected in a loop. After that, I am trying to connect the same angle points to the neighboring planes. To implement this, I start from the second plane and match every point to the point in the previous plane that has the same angle data. Then connect two points that matched. I use a for loop that implements this from the second layer to the last layer which will make sure all the points have the same angle are connected into a line. The 3D plot is completed after these implementations.

```

% close the loop by repeating the first point
x_loop = [X(idx); X(idx(1))];
y_loop = [Y(idx); Y(idx(1))];
z_loop = [Z(idx); Z(idx(1))];

```

*Figure 5: MATLAB code that repeat the first point in order to create closed loop*

```

xc = [X(idx1(a)), X(idx2(a))];
yc = [Y(idx1(a)), Y(idx2(a))];
zc = [Z(idx1(a)), Z(idx2(a))];

```

*Figure 6: MATLAB code that take one point in scan 1 and the point that have the same angle data in scan 2*

## Application Note

In the real world, lidar scanning is widely used. It can be used to measure the distance to the obstacles around it and create a 3D mapping based on the data collected. It has been widely used in automotive vehicles which is used to detect the distance to the cars or obstacles ahead and detect the distance to the wall when backing to the parking space. It will make driving much easier and safer since it can remind the driver when the distance is too short or the software system in the car can actively braking or turning to avoid collision if the distance is too short.

Moreover, it can be used to create the 3D mappings of the internal structure of the building or certain spaces more efficiently and accurately. Because when face some complex internal structures, it is hard for people to measure and draw them by hand. In addition, in some cases where people have visible issue like in the dark while the lidar will not be influenced.

## Instructions

For instructions of using this device, we assume that the user have downloaded the Keil and use the MATLAB to receive the data transmission to PC and make the 3D plot. And the hardware has been set up correctly like mounting the ToF sensor to the stepper motor.

1. Build the circuit based on the circuit schematic diagram which is Figure 13
2. Plug the microcontroller into the computer
3. Open Device Manager on the computer then look under Ports (COM & LPT) and find the XDS110 user UART and get the port number X.
4. Open MATLAB which put your corresponding "COMX" and baud rate 115200 into the statement "device = serialport("COMX", 115200)". Now, the UART communication protocol has been set up correctly between the microcontroller and the PC.
5. Change the number 3 on line 22, 301 and 344 in Keil to a number which is the number of scans you want to do in this time
6. Based on your scan place, you can change the x displacement on line 305 in Keil which is  $100 * a$  right now which means 100 mm x displacement after each scan. You can change 100 to an int number which fits the scan condition.
7. Press translate, build and load in order in Keil.
8. Open MATLAB and press run
9. Press the reset button and wait for "ToF chip booted!" to appear
10. Press on-board button PJ0
11. Wait for the scans to finish and the ToF sensor goes back to the starting point. Then the 3D plot will appear in MATLAB.

## Expected Output

The scan output is shown below in figure 7, 8, 9. In my scans, I set the x displacement between each scan is 1m. And my assign location is A which its real structure can be seen in figures 10, 11, 12. For my scan output, it is similar to the real structure, and the characteristics concluded from the scan follow the characteristics of the real place. First, from figure 7, we can conclude that the hallway is rectangular which matches the real place shown in figure 10. Also, the scan output shows that there is a longer distance measurement value in the bottom left shown in figure 8. It matches the real place shown in figure 11 which there is a hallway at there. Since the maximum

distance that the ToF sensor can measure is 4m so in the program we set all the data points that are out of range to 4m. That's why there is a longer distance measurement value. However, there is some discrepancy values shown from scan 1 to 7. It should all have the same structure as the fifth scan which is similar to a rectangle. The reason of why these discrepancies happens is because there are decorations on the ceiling (can be seen in figure 10) which will make the measurement distance increase and decrease quickly. Therefore, some discrepancies values are being measured.

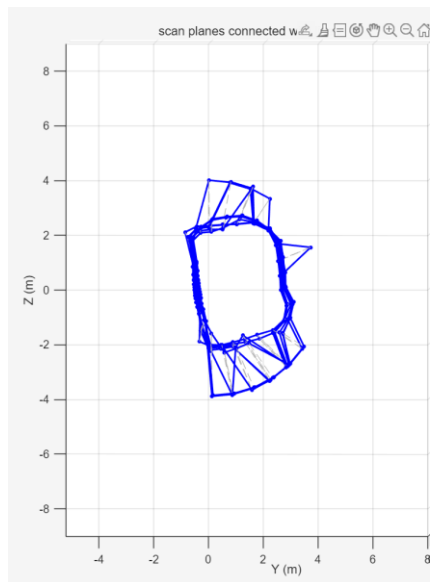


Figure 7: 3D plot front view

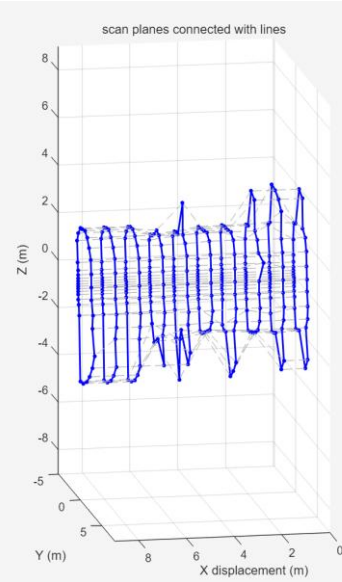


Figure 8: 3D plot general view

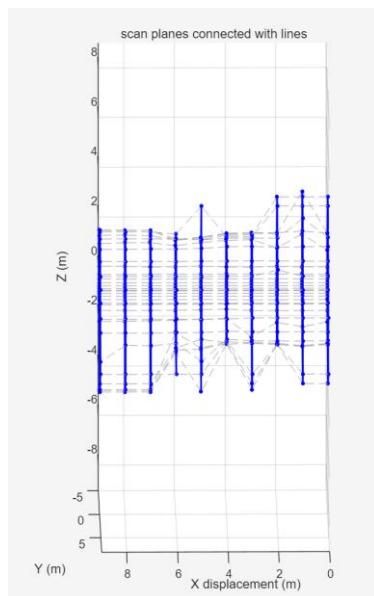


Figure 9: 3D plot side view

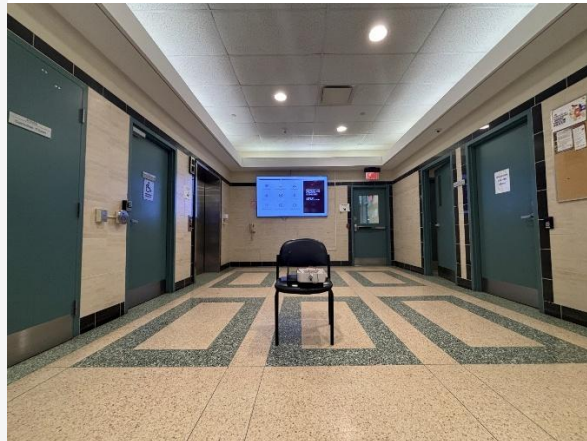


Figure 10: the photo of real scan place



Figure 11: the photo of real scan place

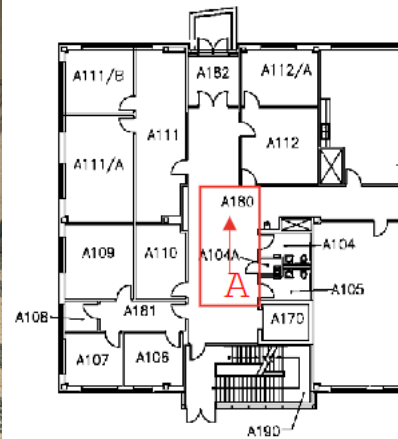


Figure 12: the top view of assigned scan place A

## Limitations

The microcontroller includes a 32-bit Cortex-M4F architecture and IEEE 754-compliant single-precision Floating-Point Unit (FPU). Although it means it is capable for the microcontroller to display and do calculations on floating-point numbers. The floating-point numbers like  $11.25^\circ$ ,  $\sin(11.25^\circ)$ , and  $\cos(11.25^\circ)$  can not be represented correctly since there are rounding errors during the calculation. In this project, it is important to make sure that the floating point is represented correctly since we need to use trigonometric functions to turn the distance data into y and z coordinates and for 3D visualization. There are several scans in our project so the rounding errors will lead to inconsistency of y, z coordinates which will make the 3D plot inaccurate. Moreover, the trigonometric calculations are more expensive than simple adding and subtract so it is inefficient to do trigonometric calculations in a microcontroller. Therefore, the microcontroller is used to collect and transmit the data to the MATLAB. Then the MATLAB will do trigonometric calculations since it has 64-bits which can do more accurate and efficient calculations.

The maximum standard serial communication rate that I can implement with the PC is 128000bps. To verify that I first open the device manager, then click ports (COM and LPT), then click the UART port used for my PC is COM4, then go to the ports settings and open the bits per second menu which shows the maximum rate my PC can achieve is 128000bps. But in this project, the baud rate is set to 115200bps.

The communication between the microcontroller and ToF sensor is through I2C. The speed of communication used in this project is 100kbps.

After reviewing the whole program, there are several components that will limit the speed which are stepper motor movement, UART transmission, and ToF sensor

measurement timing. First of all, the stepper motor movement will create the most significant speed limitation. Because in previous labs we changed the delay between each step of the stepper motor movement and I found that the motor can not move if the delay between two steps is less than around 2ms. Therefore, in one whole rotation it will take delay at least  $delay = 2ms * 2048 = 4096ms = 4.096s$ . In my program, the delay between each step is 10ms which will cause even longer delay. UART transmit the distance data and the debug messages like indicate the user which stage the program are in to the PC which will take some time during the scan. Also, we need to set delays for the ToF sensor to measure the distance data after each 11.25-degree rotation which also increases the time needed to run the program.

## Circuit Schematic

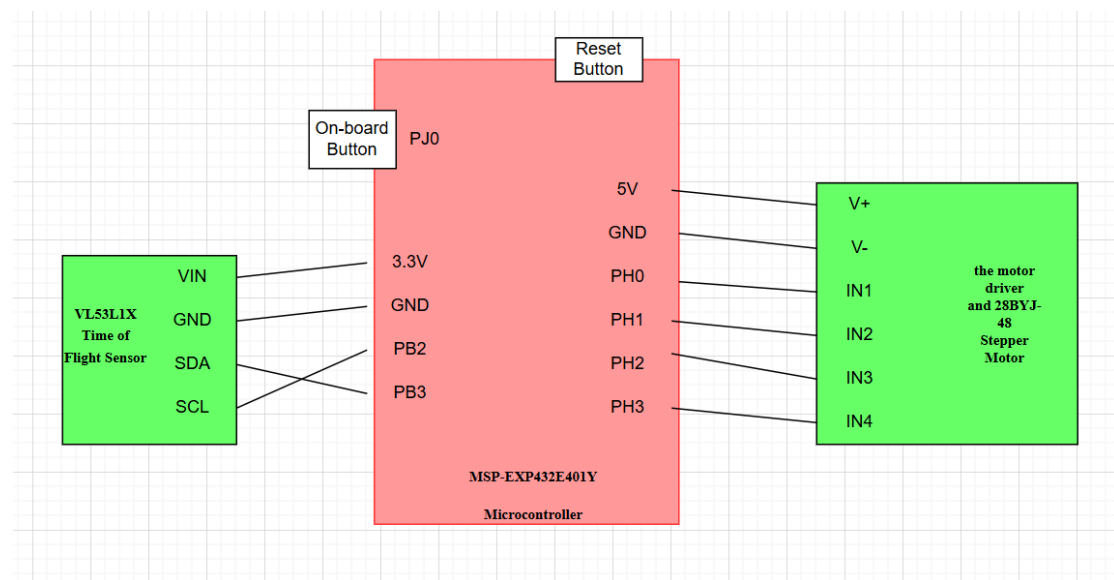


Figure 13: the circuit schematic

## Programming Logic Flowchart

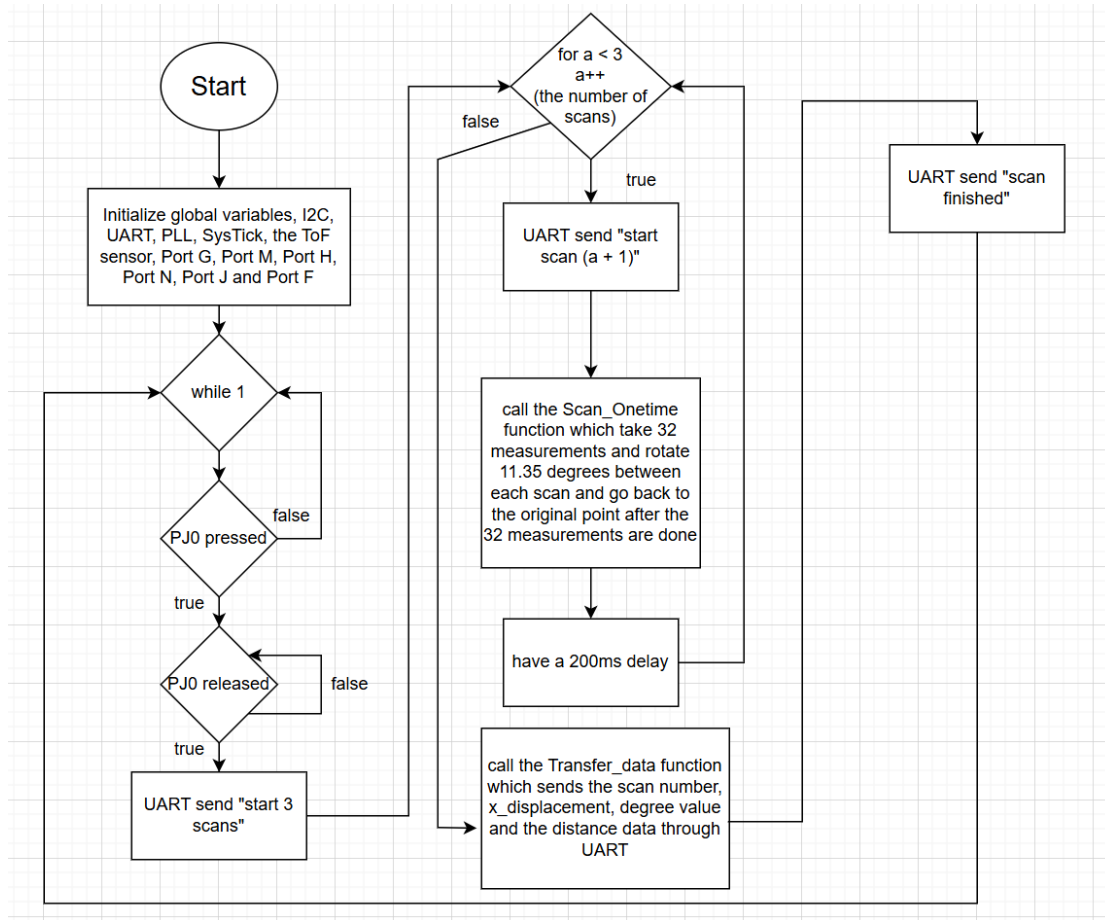


Figure 14: the programming logic flowchart

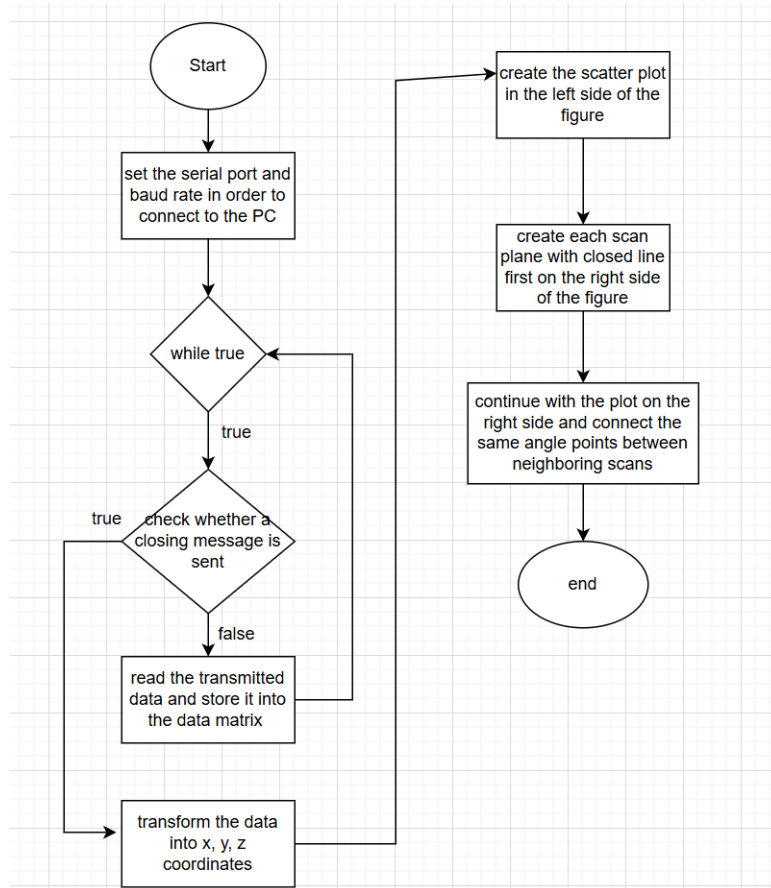


Figure 15: the MATLAB logic flowchart